Transformer (deep learning architecture)

Source: Wikipedia (en.wikipedia.org) — Educational Use

In deep learning, the transformer is a family of artificial neural network architectures based on the multi-head attention mechanism, in which input data such as text, images, or audio is converted to a sequence of numerical representations called tokens, and each token is converted into a vector through an embedding layer. At each layer, each token is then contextualized within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism, allowing the signal for key tokens to be amplified and less important tokens to be diminished. Because self-attention alone is permutation-invariant, transformers inject positional information, typically through positional encodings or learned positional embeddings, so token order can affect the output.

Because transformers do not process tokens one at a time, their computations can be parallelized across sequence positions during training more readily than those of recurrent neural networks like long short-term memory (LSTM). Later variations have been widely adopted for training large language models (LLMs) on large (language) datasets.

The original version of the transformer architecture was proposed in the 2017 paper "Attention Is All You Need" by researchers at Google. The predecessors of transformers were developed as an improvement over previous architectures for machine translation, but have found many applications since. They are used in large-scale natural language processing, computer vision (vision transformers), reinforcement learning, audio, multimodal learning, robotics, and playing chess. It has also led to the development of pre-trained systems, such as generative pre-trained transformers (GPTs).

History

Predecessors

Before transformers, recurrent neural networks (RNNs) were widely used for sequence modelling and generation. A well-cited early example was the Elman network (1990). In theory, the information from one token can propagate arbitrarily far down the sequence, but in practice the vanishing-gradient problem leaves the model's state at the end of a long sentence without precise, extractable information about preceding tokens.

A key breakthrough was LSTM (originally described in a 1995 technical report and formally published in 1997), an RNN that introduced gating mechanisms to mitigate the vanishing gradient problem, allowing efficient learning of long-sequence modelling. One key architectural element was the use of multiplicative gating units, in which the outputs of some neurons modulate the outputs of others. These multiplicative units are conceptually distinct from the additive attention mechanism later introduced for sequence-to-sequence models.

Neural networks using multiplicative units were later called sigma-pi networks or higher-order networks. LSTM became the standard architecture for long sequence modelling until the 2017 publication of transformers. However, LSTM still used sequential processing, like most other RNNs. Specifically, RNNs operate one token at a time from first to last; they cannot operate in parallel over all tokens in a sequence.

Transformers allow parallel processing of tokens during training, but standard self-attention has computational and memory costs that grow quadratically with sequence length. The fast weight

controller, proposed in 1992, used one network to generate input-dependent weights for another network. This method built on earlier work on "fast weights" and "dynamic links". A slow neural network learns by gradient descent to generate keys and values for computing the weight changes of the fast neural network which computes answers to queries. This was later shown to be equivalent to the unnormalized linear transformer.

Attention with seq2seq

The idea of encoder–decoder sequence transduction had been developed in the early 2010s; commonly cited as the originators that produced seq2seq are two concurrently published papers from 2014.

A 380M-parameter model for machine translation uses two long short-term memories (LSTM). Its architecture consists of two parts. The encoder is an LSTM that takes in a sequence of tokens and turns it into a vector. The decoder is another LSTM that converts the vector into a sequence of tokens. Similarly, another 130M-parameter model used gated recurrent units (GRU) instead of LSTM. Later research showed that GRUs are neither better nor worse than LSTMs for seq2seq.

These early seq2seq models had no attention mechanism, and the state vector is accessible only after the last word of the source text was processed. Although in theory such a vector retains the information about the whole original sentence, in practice the information is poorly preserved. This is because the input is processed sequentially by one recurrent network into a fixed-size output vector, which is then processed by another recurrent network into an output. If the input is long, then the output vector would not be able to contain all relevant information, degrading the output. As evidence, reversing the input sentence improved seq2seq translation.

The RNN search model introduced an attention mechanism to seq2seq for machine translation to solve the bottleneck problem (of the fixed-size output vector), allowing the model to process long-distance dependencies more easily. The name is because it "emulates searching through a source sentence during decoding a translation".

The relative performances were compared between global (that of RNN search) and local (sliding window) attention model architectures for machine translation, finding that mixed attention had higher quality than global attention, while local attention reduced translation time.

In 2016, Google Translate was revamped to Google Neural Machine Translation, which replaced the previous model based on statistical machine translation. The new model was a seq2seq model where the encoder and the decoder were both 8 layers of bidirectional LSTM. It took nine months to develop, and it outperformed the statistical approach, which took ten years to develop.

Parallelizing attention

Seq2seq models with attention (including self-attention) still suffered from the same issue with recurrent networks, which is that they are hard to parallelize, which prevented them from being accelerated on GPUs. In 2016, decomposable attention applied a self-attention mechanism to feedforward networks, which are easy to parallelize, and achieved state-of-the-art (SOTA) results in textual entailment with an order of magnitude fewer parameters than LSTMs. One of its authors, Jakob Uszkoreit, suspected that attention without recurrence would be sufficient for language translation, thus the title "attention is All you need". That hypothesis was against conventional wisdom at the time, and even his father Hans Uszkoreit, a well-known computational linguist, was skeptical. In the same year, self-attention (called intra-attention or intra-sentence attention) was proposed for LSTMs.

On 2017-06-12, the original (100M-parameter) encoder–decoder transformer model was published in the "Attention is All you need" paper. At the time, the focus of the research was on improving seq2seq for machine translation, by removing its recurrence to process all tokens in parallel, but preserving its dot-product attention mechanism to keep its text processing performance. This led to the introduction of a multi-head attention model that was easier to parallelize due to the use of independent heads and the lack of recurrence. Its parallelizability was an important factor to its widespread use in large neural networks.

AI boom era

In spring 2017, before the "Attention is All you need" preprint was published, one of the co-authors applied the "decoder-only" variation of the architecture to generate fictitious Wikipedia articles. Transformer architecture is now used alongside many generative models that contribute to the ongoing AI boom.

The "reference implementation" of the original Transformer was written in a TensorFlow library. In language modelling, ELMo (2018) was a bi-directional LSTM that produces contextualized word embeddings, improving upon the line of research from bag of words and word2vec. It was followed by BERT (2018), an encoder-only transformer model. In October 2019, Google started using BERT to process search queries. In 2020, Google Translate replaced the previous RNN-encoder–RNN-decoder model by a transformer-encoder–RNN-decoder model.

Starting in 2018, the OpenAI GPT series of decoder-only transformers became state of the art in natural language generation. At the end of 2022, ChatGPT, a chatbot based on a fine-tuned variant of GPT-3.5, became unexpectedly popular, triggering a boom around large language models.

Transformers have been applied in modalities beyond text. Four days after the publication of "Attention is All You Need", a multimodal transformer architecture, MultiModel, was published by most authors of that paper. Other examples include the vision transformer, which has led to developments in convolutional neural networks. Image and video generators like DALL-E (2021), Stable Diffusion 3 (2024), and Sora (2024), use transformers to analyse input data (like text prompts) by breaking it down into "tokens" and then calculating the relevance between each token using self-attention, which helps the model simulate the context and relationships within the data.

Training

Methods for stabilizing training

The plain transformer architecture had difficulty in converging. In the original paper, the authors recommended using learning rate warmup. That is, the learning rate should linearly scale up from 0 to maximal value for the first part of the training (usually recommended to be 2% of the total number of training steps), before decaying again.

A 2020 paper found that using layer normalization before (instead of after) multihead attention and feedforward layers stabilizes training, not requiring learning rate warmup. This is the "pre-LN Transformer" and is more commonly used, compared to the original "post-LN Transformer".

Regularization

The original transformer used dropout during training to reduce overfitting. Dropout was applied to the output of each sublayer before its residual connection and normalization, as well as to the sums of token embeddings and positional encodings. The original paper also used label smoothing, which replaces a certain target distribution with one that assigns a small amount of probability to alternative tokens. These techniques can improve generalization by going against overconfidence in

the training targets.

Distributed Training

Large transformer models are typically trained across multiple GPUs or other accelerators because their parameters, activations, and training data may not fit on a single device. Data parallelism copies the model across devices and splits different batches of training examples among them; gradients are then synchronized before parameters are updated. Model parallelism divides a model itself across devices. Tensor parallelism splits operations within individual layers, while pipeline parallelism assigns consecutive groups of layers to different devices.

Pretrain-finetune

Transformers typically are first pretrained by self-supervised learning on a large generic dataset, followed by supervised fine-tuning on a small task-specific dataset. The pretrain dataset is typically an unlabeled large corpus, such as The Pile. Tasks for pretraining and fine-tuning commonly include:

reading comprehension

sentiment analysis

paraphrasing

The T5 transformer report documents a large number of natural language pretraining tasks. Some examples are:

restoring or repairing incomplete or corrupted text. For example, the input, "Thank you ~~ me to your party ~~ week", might generate the output, "Thank you for inviting me to your party last week".

translation between natural languages (machine translation)

judging the pragmatic acceptability of natural language. For example, the following sentence might be judged "not acceptable", because even though it is syntactically well-formed, it is improbable in ordinary human usage: The course is jumping well.

While each of these tasks is trivial or obvious for human native speakers of the language (or languages), they have typically proved challenging for previous generations of machine learning architecture.

Knowledge distillation

Knowledge distillation is a method for training a smaller model, called a "student", to reproduce the output behavior of a larger "teacher" model. In transformer models, the student can be trained using the teacher's output probability distribution, intermediate representations, or attention patterns in addition to the original training labels. Distillation can lower computational and memory requirements of deployment while keeping most of the teacher model's performance on a specified task.

Tasks

In general, there are three classes of language modelling tasks: "masked", "autoregressive", and "prefixLM". These classes are independent of a specific modeling architecture such as transformer, but they are often discussed in the context of transformer.

In a masked task, one or more of the tokens is masked out, and the model would produce a probability distribution predicting what the masked-out tokens are based on the context. The loss

function for the task is typically sum of log-perplexities for the masked-out tokens:

Loss

=

−

∑

t

∈

masked tokens

ln

■

(

probability of

t

conditional on its context

)

{\displaystyle {\text{Loss}}=-\sum _{t\in {\text{masked tokens}}}\ln({\text{probability of }}t{\text{ conditional on its context}})}

and the model is trained to minimize this loss function. The BERT series of models are trained for masked token prediction and another task. ("Masked" as in "masked language modelling" is not "masked" as in "masked attention".)

In an autoregressive task, the entire sequence is masked at first, and the model produces a probability distribution for the first token. Then the first token is revealed and the model predicts the second token, and so on. The loss function for the task is still typically the same. The GPT series of models are trained by autoregressive tasks.

In a prefixLM task, the sequence is divided into two parts. The first part is presented as context, and the model predicts the first token of the second part. Then that would be revealed, and the model predicts the second token, and so on. The loss function for the task is still typically the same. The T5 series of models are trained by prefixLM tasks. ("PrefixLM" as in "prefix language modeling" is not "prefixLM" as in "prefix language model".)

Architecture

All transformers have the same primary components:

Tokenizers, which convert text into tokens.

Embedding layer, which converts tokens and positions of the tokens into vector representations.

Transformer layers, which carry out repeated transformations on the vector representations, extracting more and more linguistic information. These consist of alternating attention and feedforward layers. There are two major types of transformer layers: encoder layers and decoder layers, with further variants.

Un-embedding layer, which converts the final vector representations back to a probability distribution over the tokens.

The following description follows exactly the transformer as described in the original paper. There are variants, described in the following section.

By convention, we write all vectors as row vectors. For example, pushing a vector through a linear layer means multiplying it by a weight matrix on the right, as

x

W

{\displaystyle xW}

.

Tokenization

As the transformer architecture natively consists of operations over numbers (matrix multiplications, dot products, activation functions) rather than over text, there must first be a mapping from any input text to some numerical representation. This happens in three steps.

First, the input text is treated by a preprocessor, which performs both textual transformations and splits the text into coarse-grained segments called pretokens. The latter is referred to as pretokenization. Second, each pretoken is segmented further into tokens by a tokenizer that expects to only see pretokens output by its preprocessor. Each token it produces is a string of one or more characters belonging to a finite set of strings called the vocabulary

V

{\displaystyle V}

. Third, because the vocabulary is finite and known beforehand, each token can be assigned an integer identifier, and this mapping is applied to the sequence of tokens to represent any input text as a numerical sequence. Since this mapping is bijective, the output side can produce a sequence of integer identifiers which can then be turned back into tokens. After undoing some of the preprocessing, the result is again legible text.

Training a tokenizer (sometimes referred to as vocabularization) means finding a suitable vocabulary

V

{\displaystyle V}

, but also learning how to use it, since any given string

s

{\displaystyle s}

of length

|

s

|

{\displaystyle |s|}

has

2

|

s

|

–

1

{\displaystyle 2^{|s|-1}}

hypothetical segmentations, some of which containing segments that are not in the vocabulary. The most important hyperparameter during vocabularization is the vocabulary size

|

V

|

{\displaystyle |V|}

: when it is small, the learned vocabulary generally consists of characters and smaller strings, and words will be segmented into many tokens. At larger sizes, it becomes affordable to dedicate tokens to full words, although depending on the preprocessor and tokenizer, it is not necessarily the case

that large vocabularies will always use the largest token(s) available to segment a word.

Because tokens are not always full words, they may also be referred to as subwords and tokenization algorithms may be referred to as subword tokenizers. This is also to differentiate these systems from traditional terminology used in older information retrieval and natural language processing systems, where "tokenization" was used to denote what is today called "pretokenization" (very crudely: splitting into words). In tokenizers that produce tokens that are not part of the vocabulary, a special token that does belong to the vocabulary is used as a generic stand-in, written as "[UNK]" for "unknown". In principle, any string could be hidden by such an [UNK]. Indeed, in information retrieval, pretokenizers were themselves used as tokenizers (and also called "tokenizers") with a word-level vocabulary that contained an [UNK].

Commonly used subword tokenization algorithms are byte pair encoding (BPE) and the unigram language model (ULM), which each include a vocabularization algorithm and a dedicated segmentation algorithm. There also exist several segmentation algorithms that require no learning and can be applied given a vocabulary (produced by BPE or ULM, for example), like greedily recognising tokens in a pretoken by moving through it left-to-right. Well-known software implementations of subword tokenizers are Hugging Face's tokenizers Python package implemented in Rust, and the sentencepiece Python package implemented in C++. The latter package is named as such because one of its configuration options allows disabling the built-in pretokenizer, hence effectively making entire sentences a pretoken and thus having the tokenizer see entire sentences, rather than individual words.

Embedding

Each integer token identifier is converted into an embedding vector via a lookup table. Equivalently stated, it multiplies a one-hot representation of the token identifier by an embedding matrix

M

{\displaystyle M}

. For example, if the input token's identifier is

3

{\displaystyle 3}

, then the one-hot representation is

[

0

,

0

,

0

,

1

,

0

,

0

,

…

]

{\displaystyle [0,0,0,1,0,0,\dots]}

, and its embedding vector is

E

m

b

e

d

(

3

)

=

[

0

,

0

,

0

,

1

,

0

,

0

,

…

]

M

{\displaystyle \mathrm {Embed} (3)=[0,0,0,1,0,0,\dots]M}

The token embedding vectors are added to their respective positional encoding vectors (see below), producing the sequence of input vectors.

The dimension of an embedding vector is called hidden size or embedding size and written as

d

emb

{\displaystyle d_{\text{emb}}}

. This size is written as

d

model

{\displaystyle d_{\text{model}}}

in the original transformer paper.

Un-embedding

An un-embedding layer is almost the reverse of an embedding layer. Whereas an embedding layer converts a token identifier into a vector, an un-embedding layer converts a vector into a probability distribution over tokens.

The un-embedding layer is a linear-softmax layer:

U

n

E

m

b

e

d

(

x

)

=

s

o

f

t

m

a

x

(

x

W

+

b

)

{\displaystyle \mathrm {UnEmbed} (x)=\mathrm {softmax} (xW+b)}

The matrix has shape

(

d

emb

,

|

V

|

)

{\displaystyle (d_{\text{emb}},|V|)}

. Some architectures use the transpose of the embedding matrix

M

{\displaystyle M}

as the un-embedding matrix

W

{\displaystyle W}

in order to avoid needing double the amount of embedding-related parameters and to avoid divergence during training. This practice is called weight tying.

Positional encoding

A positional encoding is a fixed-size vector representation of the relative positions of tokens within a sequence: it provides the transformer model with information about where the words are in the input sequence. This induces a bias towards the order of the input sequence, so that, for example, the input sequence "man bites dog" is processed differently from "dog bites man".

The positional encoding is defined as a function of type

f

:

R

→

R

d

{\displaystyle f:\mathbb {R} \to \mathbb {R} ^{d}}

, where

d

{\displaystyle d}

is a positive even integer. The full positional encoding defined in the original paper is:

$$
(
f
(
t
)
2
k
,
f
(
t
)
2
k
+
1
)
)
=
(
\sin
\blacksquare
(
\theta
)
,
\cos
\blacksquare
(
\theta
)
)
$$

∀

k

∈

{

0

,

1

,

…

,

d

/

2

–

1

}

{\displaystyle (f(t)_{2k},f(t)_{2k+1})=(\sin(\theta),\cos(\theta))\quad \forall k\in \{0,1,\ldots ,d/2-1\}}

where

θ

=

t

r

k

,

r

=

N

2

/

d

{\displaystyle \theta ={\frac {t}{r^{k}}},r=N^{2/d}}

.

Here,

N

{\displaystyle N}

is a free parameter that should be significantly larger than the biggest

k

{\displaystyle k}

that would be input into the positional encoding function. The original paper uses

N

=

10000

{\displaystyle N=10000}

.

The function is in a simpler form when written as a complex function of type

f

:

R

→

C

d

/

2

{\displaystyle f:\mathbb {R} \to \mathbb {C} ^{d/2}}

f
(
t
)
=
(
e
i
t
/
r
k
)
k
=
0
,
1
,
…
,
d
2
−
1

{\displaystyle f(t)=\left(e^{it/r^{k}}\right)_{k=0,1,\ldots ,{\frac {d}{2}}-1}}

where

r

=

N

2

/

d

{\displaystyle r=N^{2/d}}

.

The main reason for using this positional encoding function is that using it, shifts are linear transformations:

f

(

t

+

Δ

t

)

=

d

i

a

g

(

f

(

Δ

t

)

)

f

(

t

)

{\displaystyle f(t+\Delta t)=\mathrm {diag} (f(\Delta t))f(t)}

where

Δ

t

∈

R

{\displaystyle \Delta t\in \mathbb {R} }

is the distance one wishes to shift. This allows the transformer to take any encoded position, and find the encoding of the position n-steps-ahead or n-steps-behind, by a matrix multiplication.

By taking a linear sum, any convolution can also be implemented as linear transformations:

Σ

j

c

j

f

(

t

+

Δ

t

j

)

=

(

∑

j

c

j

d

i

a

g

(

f

(

Δ

t

j

)

)

)

f

(

t

)

{\displaystyle \sum _{j}c_{j}f(t+\Delta t_{j})=\left(\sum _{j}c_{j}\,\mathrm {diag} (f(\Delta t_{j}))\right)f(t)}

for any constants

c

j

{\displaystyle c_{j}}

. This allows the transformer to take any encoded position and find a linear sum of the encoded locations of its neighbors. This sum of encoded positions, when fed into the attention mechanism, would create attention weights on its neighbors, much like what happens in a convolutional neural network language model. In the author's words, "we hypothesized it would allow the model to easily learn to attend by relative position."

In typical implementations, all operations are done over the real numbers, not the complex numbers, but since complex multiplication can be implemented as real 2-by-2 matrix multiplication, this is a mere notational difference.

Encoder–decoder (overview)

Like earlier seq2seq models, the original transformer model used an encoder–decoder architecture. The encoder consists of encoding layers that process all the input tokens together one layer after another, while the decoder consists of decoding layers that iteratively process the encoder's output and the decoder's output tokens so far.

The purpose of each encoder layer is to create contextualized representations of the tokens, where each representation corresponds to a token that "mixes" information from other input tokens via self-attention mechanism. Each decoder layer contains two attention sublayers: (1) cross-attention for incorporating the output of encoder (contextualized input token representations), and (2) self-attention for "mixing" information among the input tokens to the decoder (i.e. the tokens generated so far during inference time).

Both the encoder and decoder layers have a feed-forward neural network for additional processing of their outputs and contain residual connections and layer normalization steps. These feed-forward layers contain most of the parameters in a transformer model.

Feedforward network

The feedforward network (FFN) modules in a transformer are 2-layered multilayer perceptrons:

F

F

N

(

x

)

=

φ

(

x

W

(

1

)

+

b

(

1

)

)

W

(

2

)

+

b

(

2

)

{\displaystyle \mathrm {FFN} (x)=\phi (xW^{(1)}+b^{(1)})W^{(2)}+b^{(2)}}

where

W

(

1

)

{\displaystyle W^{(1)}}

and

W

(
2
)

{\displaystyle W^{(2)}}

are weight matrices and

b

(
1
)

{\displaystyle b^{(1)}}

and

b

(
2
)

{\displaystyle b^{(2)}}

are bias vectors, and

φ

{\displaystyle \phi }

is its activation function. The original transformer used ReLU activation.

The number of neurons in the middle layer is called intermediate size (GPT), filter size (BERT), or feedforward size (BERT). It is typically larger than the embedding size. For example, in both GPT-2 series and BERT series, the intermediate size of a model is 4 times its embedding size:

d

ffn

=

4

d

emb

{\displaystyle d_{\text{ffn}}=4d_{\text{emb}}}

.

Scaled dot-product attention

Attention head

The attention mechanism used in the transformer architecture are scaled dot-product attention units. For each unit, the transformer model learns three weight matrices: the query weights

W

Q

{\displaystyle W^{Q}}

, the key weights

W

K

{\displaystyle W^{K}}

, and the value weights

W

V

{\displaystyle W^{V}}

.

The module takes three sequences, a query sequence, a key sequence, and a value sequence. The query sequence is a sequence of length

■

seq, query

{\displaystyle \ell _{\text{seq, query}}}

, and each entry is a vector of dimension

d

emb, query

{\displaystyle d_{\text{emb, query}}}

. Similarly for the key and value sequences.

For each vector

x

i

,

query

{\displaystyle x_{i,{\text{query}}}}

in the query sequence, it is multiplied by a matrix

W

Q

{\displaystyle W^{Q}}

to produce a query vector

q

i

=

x

i

,

query

W

Q

{\displaystyle q_{i}=x_{i,{\text{query}}}W^{Q}}

. The matrix of all query vectors is the query matrix:

Q

=

X

query

W

Q

{\displaystyle Q=X_{\text{query}}W^{Q}}

Similarly, we construct the key matrix

K

=

X

key

W

K

{\displaystyle K=X_{\text{key}}W^{K}}

and the value matrix

V

=

X

value

W

V

{\displaystyle V=X_{\text{value}}W^{V}}

.

It is usually the case that all

W

Q

,

W

K

,

W

V

{\displaystyle W^{Q},W^{K},W^{V}}

are square matrices, meaning

d

emb, query

=

d

query

{\displaystyle d_{\text{emb, query}}=d_{\text{query}}}

, etc.

Attention weights are calculated using the query and key vectors: the attention weight

a

i

j

{\displaystyle a_{ij}}

from token

i

{\displaystyle i}

to token

j

{\displaystyle j}

is the dot product between

q

i

{\displaystyle q_{i}}

and

k

j

{\displaystyle k_{j}}

. The attention weights are divided by the square root of the dimension of the key vectors,

d

k

{\displaystyle {\sqrt {d_{k}}}}

, which stabilizes gradients during training, and passed through a softmax which normalizes the weights. The fact that

W

Q

{\displaystyle W^{Q}}

and

W

K

{\displaystyle W^{K}}

are different matrices allows attention to be non-symmetric: if token

i

{\displaystyle i}

attends to token

j

{\displaystyle j}

(i.e.

q

i

.

k

j

{\displaystyle q_{i}\cdot k_{j}}

is large), this does not necessarily mean that token

j

{\displaystyle j}

will attend to token

i

{\displaystyle i}

(i.e.

q

j

.

k

i

{\displaystyle q_{j}\cdot k_{i}}

could be small). The output of the attention unit for token

i

{\displaystyle i}

is the weighted sum of the value vectors of all tokens, weighted by

a

i

j

{\displaystyle a_{ij}}

, the attention from token

i

{\displaystyle i}

to each token.

The attention calculation for all tokens can be expressed as one large matrix calculation using the softmax function, which is useful for training due to computational matrix operation optimizations that quickly compute matrix operations. The matrices

Q

{\displaystyle Q}

,

K

{\displaystyle K}

and

V

{\displaystyle V}

are defined as the matrices where the

i

{\displaystyle i}

th rows are vectors

q

i

{\displaystyle q_{i}}

,

k

i

{\displaystyle k_{i}}

, and

v

i

{\displaystyle v_{i}}

respectively. Then we can represent the attention as

Attention

(

Q

,

K

,

V

)

=

softmax

(

Q

K

T

d

k

)

V

{\displaystyle {\begin{aligned}{\text{Attention}}(Q,K,V)={\text{softmax}}\left({\frac {QK^{\mathrm {T} }}{\sqrt {d_{k}}}}\right)V\end{aligned}}}

where the softmax is applied over each of the rows of the matrix.

The number of dimensions in a query vector is query size

d

query

{\displaystyle d_{\text{query}}}

and similarly for the key size

d

key

{\displaystyle d_{\text{key}}}

and value size

d

value

{\displaystyle d_{\text{value}}}

. The output dimension of an attention head is its head dimension

d

head

{\displaystyle d_{\text{head}}}

. The attention mechanism requires the following three equalities to hold:

■

seq, key

=

■

seq, value

,

d

query

=

d

key

,

d

value

=

d

head

{\displaystyle \ell _{\text{seq, key}}=\ell _{\text{seq, value}}},\;d_{\text{query}}=d_{\text{key}},\;d_{\text{value}}=d_{\text{head}}}

but is otherwise unconstrained.

If the attention head is used in a self-attention fashion, then

X

query

=

X

key

=

X

value

{\displaystyle X_{\text{query}}=X_{\text{key}}=X_{\text{value}}}

. If the attention head is used in a cross-attention fashion, then usually

X

query

≠

X

key

=

X

value

{\displaystyle X_{\text{query}}\neq X_{\text{key}}=X_{\text{value}}}

. It is theoretically possible for all three to be different, but that is rarely the case in practice.

Multihead attention

One set of

(

W

Q

,

W

K

,

W

V

)

{\displaystyle \left(W^{Q},W^{K},W^{V}\right)}

matrices is called an attention head, and each layer in a transformer model has multiple attention heads. While each attention head attends to the tokens that are relevant to each token, multiple attention heads allow the model to do this for different definitions of "relevance". Specifically, the query and key projection matrices,

W

Q

{\displaystyle W^{Q}}

and

W

K

{\displaystyle W^{K}}

, which are involved in the attention score computation, defines the "relevance". Meanwhile, the value projection matrix

W

V

{\displaystyle W^{V}}

, in combination with the part of the output projection matrix

W

O

{\displaystyle W^{O}}

, determines how the attended tokens influence what information is passed to subsequent layers and ultimately the output logits. In addition, the scope of attention, or the range of token relationships captured by each attention head, can expand as tokens pass through successive layers. This allows the model to capture more complex and long-range dependencies in deeper layers. Many transformer attention heads encode relevance relations that are meaningful to humans. For example, some attention heads can attend mostly to the next word, while others mainly attend from verbs to their direct objects. The computations for each attention head can be performed in parallel, which allows for fast processing. The outputs for the attention layer are concatenated to pass into the feedforward neural network layers.

Concretely, let the multiple attention heads be indexed by

i

{\displaystyle i}

, then we have

MultiheadAttention

(

Q

,

K

,

V

)

=

Concat

i

∈

[

n

heads

]

(

Attention

(

X

W

i

Q

,

X

W

i

K

,

X

W

i

V

)

)

W

O

{\displaystyle {\text{MultiheadAttention}}(Q,K,V)={\text{Concat}}_{i\in [n_{\text{heads}}]}({\text{Attention}}(XW_{i}^{Q},XW_{i}^{K},XW_{i}^{V}))W^{O}}

where the matrix

X

{\displaystyle X}

is the concatenation of word embeddings, and the matrices

W

i

Q

,

W

i

K

,

W

i

V

{\displaystyle W_{i}^{Q},W_{i}^{K},W_{i}^{V}}

are "projection matrices" owned by individual attention head

i

{\displaystyle i}

, and

W

O

{\displaystyle W^{O}}

is a final projection matrix owned by the whole multihead attention head.

It is theoretically possible for each attention head to have a different head dimension

d

head

{\displaystyle d_{\text{head}}}

, but that is rarely the case in practice.

As an example, in the smallest GPT-2 model, there are only self-attention mechanisms. It has the following dimensions:

d

emb

=

768

,

n

head

=

12

,

d

head

=

64

{\displaystyle d_{\text{emb}}=768,n_{\text{head}}=12,d_{\text{head}}=64}

Since

12

×

64

=

768

{\displaystyle 12\times 64=768}

, its output projection matrix

W

O

∈

R

(

12

×

64

)

×

768

{\displaystyle W^{O}\in \mathbb {R} ^{(12\times 64)\times 768}}

is a square matrix.

Masked attention

The transformer architecture is constructed to calculate output tokens iteratively. Assuming

t

=

0

{\displaystyle t=0}

refers to the calculation of the first output token

i

=

0

{\displaystyle i=0}

, for step

t

>

0

{\displaystyle t>0}

, the output token

i

=

0

{\displaystyle i=0}

shall remain constant. This ensures properties of the model similar to autoregressive models. Therefore, at every time step

t

{\displaystyle t}

, the calculation for all outputs

i

{\displaystyle i}

should not have access to tokens at position

j

{\displaystyle j}

for

j

>=

i

{\displaystyle j>=i}

(as it naturally is the case for time step

t

=

i

{\displaystyle t=i}

, when tokens

j

>

t

{\displaystyle j>t}

are not yet calculated). This behavior may be accomplished before the softmax stage by adding a mask matrix

M

{\displaystyle M}

that is

–

∞

{\displaystyle -\infty }

at entries where the attention link must be cut, and

0

{\displaystyle 0}

at other places:

MaskedAttention

(

Q

,

K

,

V

)

=

softmax

(

M

+

Q

K

T

d

k

)

V

{\displaystyle {\begin{aligned}{\text{MaskedAttention}}(Q,K,V)={\text{softmax}}\left(M+{\frac {QK^{\mathrm {T} }}{\sqrt {d_{k}}}}\right)V\end{aligned}}}

The following matrix is commonly used in decoder self-attention modules, called "causal masking":

M

causal

=

[

0

–

∞

–

∞

…

–

∞

0

0

–

∞

…

–

∞

0

0

0

…

–

∞

■

■

■

■

■

0

0

0

…

0

]

{\displaystyle M_{\text{causal}}={\begin{bmatrix}0&-\infty &-\infty &\dots &-\infty \\0&0&-\infty &\dots &-\infty \\0&0&0&\dots &-\infty \\\vdots &\vdots &\vdots &\ddots &\vdots \\0&0&0&\dots &0\end{bmatrix}}}

In words, it means that each token can pay attention to itself, and every token before it, but not any after it. A non-masked attention module can be thought of as a masked attention module where the mask has all entries zero. As an example of an uncommon use of mask matrix, the XLNet considers all masks of the form

P

M

causal

P

_

1

{\displaystyle PM_{\text{causal}}P^{-1}}

, where

P

{\displaystyle P}

is a random permutation matrix.

Encoder

An encoder consists of an embedding layer, followed by multiple encoder layers.

Each encoder layer consists of two major components: a self-attention mechanism and a feed-forward layer. It takes an input as a sequence of input vectors, applies the self-attention

mechanism, to produce an intermediate sequence of vectors, then applies the feed-forward layer for each vector individually. Schematically, we have:

given input vectors

h

0

,

h

1

,

…

combine them into a matrix

H

=

[

h

0

h

1

■

]

EncoderLayer

(

H

)

=

[

Ffn

(

MultiheadAttention

(

H

,

H

,

H

)

0

)

Ffn

(

MultiheadAttention

(

H

,

H

,

H

)

1

)

■

]

{\displaystyle {\begin{aligned}{\text{given input vectors }}&h_{0},h_{1},\dots \\{\text{combine them into a matrix }}H&={\begin{bmatrix}h_{0}\\h_{1}\\\vdots \end{bmatrix}}\\{\text{EncoderLayer}}(H)&={\begin{bmatrix}{\text{FFN}}({\text{MultiheadAttention}}(H,H,H)_{0})\\{\text{FFN}}({\text{MultiheadAttention}}(H,H,H)_{1})\\\vdots \end{bmatrix}}\\\end{aligned}}}

where

Ffn

{\displaystyle {\text{FFN}}}

stands for "feed-forward network". We can more succinctly write it as

EncoderLayer

(

H

)

=

Ffn

(

MultiheadAttention

(

H

,

H

,

H

)

)

{\displaystyle {\text{EncoderLayer}}(H)={\text{FFN}}({\text{MultiheadAttention}}(H,H,H))}

with the implicit convention that the

Ffn

{\displaystyle {\text{FFN}}}

is applied to each row of the matrix individually.

The encoder layers are stacked. The first encoder layer takes the sequence of input vectors from the embedding layer, producing a sequence of vectors. This sequence of vectors is processed by the second encoder, and so on. The output from the final encoder layer is then used by the decoder.

As the encoder processes the entire input all at once, every token can attend to every other token (all-to-all attention), so there is no need for causal masking.

Decoder

A decoder consists of an embedding layer, followed by multiple decoder layers, followed by an un-embedding layer.

Each decoder consists of three major components: a causally masked self-attention mechanism, a cross-attention mechanism, and a feed-forward neural network. The decoder functions in a similar fashion to the encoder, but an additional attention mechanism is inserted which instead draws relevant information from the encodings generated by the encoders. This mechanism can also be called the encoder–decoder attention.

Like the first encoder, the first decoder takes positional information and embeddings of the output sequence as its input, rather than encodings. The transformer must not use the current or future output to predict an output, so the output sequence must be partially masked to prevent this reverse information flow. This allows for autoregressive text generation. For decoding, all-to-all attention is inappropriate, because a token cannot attend to tokens not yet generated. Thus, the self-attention module in the decoder is causally masked.

In contrast, the cross-attention mechanism attends to the output vectors of the encoder, which is computed before the decoder starts decoding. Consequently, there is no need for masking in the cross-attention mechanism.

Schematically, we have:

$$H' = \text{MaskedMultiheadAttention}(H, H, H)$$

$$\text{DecoderLayer}(H) = \text{Ffn}(\text{MultiheadAttention}($$

H

′

,

H

E

,

H

E

)

)

{\displaystyle {\begin{aligned}H'&={\text{MaskedMultiheadAttention}}(H,H,H)\\{\text{DecoderLayer}}(H)&={\text{FFN}}({\text{MultiheadAttention}}(H',H^{E},H^{E}))\end{aligned}}}

where

H

E

{\displaystyle H^{E}}

is the matrix with rows being the output vectors from the encoder.

The last decoder is followed by a final un-embedding layer to produce the output probabilities over the vocabulary. Then, one of the tokens is sampled according to the probability, and the decoder can be run again to produce the next token, etc., autoregressively generating output text.

Full transformer architecture

Sublayers

Each encoder layer contains 2 sublayers: the self-attention and the feedforward network. Each decoder layer contains 3 sublayers: the causally masked self-attention, the cross-attention, and the feedforward network.

The final points of detail are the residual connections and layer normalization, (denoted as "LayerNorm", or "LN" in the following), which while conceptually unnecessary, are necessary for numerical stability and convergence.

The residual connections are introduced to avoid vanishing gradient issues and stabilize the training process. They can be expressed by

x

■

F

(

x

)

+

x

{\displaystyle x\mapsto F(x)+x}

, where

F

{\displaystyle F}

is a given component of the transformer. Adding the input

x

{\displaystyle x}

can preserve the input information and avoid issues when the gradient of

F

(

x

)

{\displaystyle F(x)}

is close to zero.

Similarly to how the feedforward network modules are applied individually to each vector, the LayerNorm is also applied individually to each vector.

There are two common conventions in use: the post-LN and the pre-LN convention. In the post-LN convention, the output of each sublayer is

L

a

y

e

r

N

o

r

m

(

x

+

S

u

b

l

a

y

e

r

(

x

)

)

{\displaystyle \mathrm {LayerNorm} (x+\mathrm {Sublayer} (x))}

where

S

u

b

l

a

y

e

r

(

x

)

{\displaystyle \mathrm {Sublayer} (x)}

is the function implemented by the sublayer itself.

In the pre-LN convention, the output of each sublayer is

x

+

S

u

b

l

a

y

e

r

(

L

a

y

e

r

N

o

r

m

(

x

)

)

{\displaystyle x+\mathrm {Sublayer} (\mathrm {LayerNorm} (x))}

The original 2017 transformer used the post-LN convention. It was difficult to train and required careful hyperparameter tuning and a "warm-up" in learning rate, where it starts small and gradually increases. The pre-LN convention, proposed several times in 2018, was found to be easier to train, requiring no warm-up, leading to faster convergence.

Pseudocode

The following is the pseudocode for a standard pre-LN encoder–decoder transformer, adapted from Formal Algorithms for Transformers.

input: Encoder input t_e

Decoder input t_d

output: Array of probability distributions, with shape (decoder vocabulary size x length(decoder output sequence))

/* encoder */

z_e ← encoder.tokenizer(t_e)

for each t in 1:length(z_e) do

z_e[t] ← encoder.embedding(z_e[t]) + encoder.positional_embedding(t)

for each l in 1:length(encoder.layers) do

layer ← encoder.layers[l]

/* first sublayer */

z_e_copy ← copy(z_e)

for each t in 1:length(z_e) do

```
z_e[t] ← layer.layer_norm(z_e[t])
z_e ← layer.multihead_attention(z_e, z_e, z_e)
for each t in 1:length(z_e) do
z_e[t] ← z_e[t] + z_e_copy[t]
/* second sublayer */
z_e_copy ← copy(z_e)
for each t in 1:length(z_e) do
z_e[t] ← layer.layer_norm(z_e[t])
z_e ← layer.feedforward(z_e)
for each t in 1:length(z_e) do
z_e[t] ← z_e[t] + z_e_copy[t]
for each t in 1:length(z_e) do
z_e[t] ← encoder.final_layer_norm(z_e[t])
/* decoder */
z_d ← decoder.tokenizer(t_d)
for each t in 1:length(z_d) do
z_d[t] ← decoder.embedding(z_d[t]) + decoder.positional_embedding(t)
for each l in 1:length(decoder.layers) do
layer ← decoder.layers[l]
/* first sublayer */
z_d_copy ← copy(z_d)
for each t in 1:length(z_d) do
z_d[t] ← layer.layer_norm(z_d[t])
z_d ← layer.masked_multihead_attention(z_d, z_d, z_d)
for each t in 1:length(z_d) do
z_d[t] ← z_d[t] + z_d_copy[t]
/* second sublayer */
z_d_copy ← copy(z_d)
for each t in 1:length(z_d) do
z_d[t] ← layer.layer_norm(z_d[t])
z_d ← layer.multihead_attention(z_d, z_e, z_e)
for each t in 1:length(z_d) do
z_d[t] ← z_d[t] + z_d_copy[t]
```

/* third sublayer */

z_d_copy ← copy(z_d)

for each t in 1:length(z_d) do

z_d[t] ← layer.layer_norm(z_d[t])

z_d ← layer.feedforward(z_d)

for each t in 1:length(z_d) do

z_d[t] ← z_d[t] + z_d_copy[t]

z_d ← decoder.final_layer_norm(z_d)

output_distributions ← []

for each t in 1:length(z_d) do

output_distributions.append(decoder.unembed(z_d[t]))

return output_distributions

Terminology

The transformer architecture, being modular, allows variations. Several common variations are described here.

An "encoder-only" transformer applies the encoder to map an input text into a sequence of vectors that represent the input text. This is usually used for text embedding and representation learning for downstream applications. BERT is encoder-only. They are less often used currently, as they were found to be not significantly better than training an encoder–decoder transformer, then taking just the encoder. They are also referred to as "all-to-all" or "BERT-like".

A "decoder-only" transformer is not literally decoder-only, since without an encoder, the cross-attention mechanism has nothing to attend to. Thus, the decoder layers in a decoder-only transformer are composed of just two sublayers: the causally masked self-attention, and the feedforward network. This is usually used for text generation and instruction following. The models in the GPT series and Chinchilla series are decoder-only. They are also referred to as "autoregressive" or "causal".

An "encoder–decoder" transformer is generally the same as the original transformer, with 2 sublayers per encoder layer and 3 sublayers per decoder layer, etc. They might have minor architectural improvements, such as alternative activation functions, changing the location of normalization, etc. This is also usually used for text generation and instruction following. The models in the T5 series are encoder–decoder.

A "prefixLM" (prefix language model) is a decoder-only architecture, but with prefix masking, which is different from causal masking. Specifically, it has mask of the form

M

prefixLM

=

[

0

−

∞

0

M

causal

]

{\displaystyle M_{\text{prefixLM}}={\begin{bmatrix}\mathbf {0} &-\infty \\\mathbf {0} &M_{\text{causal}}\end{bmatrix}}}

where the first columns correspond to the "prefix", and the subsequent columns correspond to the autoregressively generated text based on the prefix. They resemble encoder–decoder models, but has less "sparsity". Such models are rarely used, though they are cited as theoretical possibilities and benchmarked comparisons.

There are also mixed seq2seq models. For example, in 2020, Google Translate replaced the previous RNN-encoder–RNN-decoder model with a transformer-encoder–RNN-decoder model, as transformer-based decoders did not appear to significantly increase quality unlike the encoder, while the RNN decoder was much faster.

Subsequent work

Alternative activation functions

The original transformer uses ReLU activation function. Other activation functions were developed. The Llama series and PaLM used SwiGLU; both GPT-1 and BERT used GELU.

Alternative activation functions are often used in combination with Gated Linear Units in the feedforward module.

Alternative normalizations

The normalization used in the transformer can be different from LayerNorm. One example is RMSNorm which is used in the Llama series. Other examples include ScaleNorm and FixNorm.

Alternative positional encodings

Transformers may use other positional encoding methods than sinusoidal.

The original transformer paper reported using a learned positional encoding, but finding it not superior to the sinusoidal one. Later, found that causal masking itself provides enough signal to a transformer decoder that it can learn to implicitly perform absolute positional encoding without the positional encoding module.

RoPE

RoPE (rotary positional embedding), is best explained by considering a list of 2-dimensional vectors

[

(

x

1

(

1

)

,

x

1

(

2

)

)

,

(

x

2

(

1

)

,

x

2

(

2

)

)

,

(

x

3

(

1

)

,

x

3

(

2

)

)

,

.

.

.

]

{\displaystyle [(x_{1}^{(1)},x_{1}^{(2)}),(x_{2}^{(1)},x_{2}^{(2)}),(x_{3}^{(1)},x_{3}^{(2)}),...]}

. Now pick some angle

θ

{\displaystyle \theta }

. Then RoPE encoding is

RoPE

(

x

m

(

1

)

,

x

m

(

2

)

,

m

)

=

(

cos

■

m

θ

–

sin

■

m

θ

sin

■

m

θ

cos

■

m

θ

)

(

x

m

(

1

)

x

m

(

2

)

)

=

(

x

m

(

1

)

cos

■

m

θ

–

x

m

(

2

)

sin

■

m

θ

x

m

(

2

)

cos

■

m

θ

+

x

m

(

1

)

sin

■

m

θ

)

{\displaystyle {\text{RoPE}}{\big (}x_{m}^{(1)},x_{m}^{(2)},m{\big)}={\begin{pmatrix}\cos m\theta &-\sin m\theta \\\sin m\theta &\cos m\theta \end{pmatrix}}{\begin{pmatrix}x_{m}^{(1)}\\x_{m}^{(2)}\\\end{pmatrix}}={\begin{pmatrix}x_{m}^{(1)}\cos m\theta -x_{m}^{(2)}\sin m\theta \\x_{m}^{(2)}\cos m\theta +x_{m}^{(1)}\sin m\theta \\\end{pmatrix}}}

Equivalently, if we write the 2-dimensional vectors as complex numbers

z

m

:=

x

m

(

1

)

+

i

x

m

(

2

)

{\displaystyle z_{m}:=x_{m}^{(1)}+ix_{m}^{(2)}}

, then RoPE encoding is just multiplication by an angle:

RoPE

(

z

m

,

m

)

=

e

i

m

θ

z

m

{\displaystyle {\text{RoPE}}{\big (}z_{m},m{\big)}=e^{im\theta }z_{m}}

For a list of

2

n

{\displaystyle 2n}

-dimensional vectors, a RoPE encoder is defined by a sequence of angles

θ

(

1

)

,

.

.

.

,

θ

(

n

)

{\displaystyle \theta ^{(1)},...,\theta ^{(n)}}

. Then the RoPE encoding is applied to each pair of coordinates.

The benefit of RoPE is that the dot-product between two vectors depends on their relative location only:

RoPE

(

x

,

m

)

T

RoPE

(

y

,

n

)

=

RoPE

(

x

,

m

+

k

)

T

RoPE

(

y

,

n

+

k

)

{\displaystyle {\text{RoPE}}{\big (}x,m{\big)}^{T}{\text{RoPE}}{\big (}y,n{\big)}={\text{RoPE}}{\big (}x,m+k{\big)}^{T}{\text{RoPE}}{\big (}y,n+k{\big)}}

for any integer

k

{\displaystyle k}

.

ALiBi

ALiBi (Attention with Linear Biases) is not a replacement for the positional encoder on the original transformer. Instead, it is an additional positional encoder that is directly plugged into the attention mechanism. Specifically, the ALiBi attention mechanism is

Attention

(

Q

,

K

,

V

)

=

softmax

(

Q

K

T

d

k

+

s

B

)

V

{\displaystyle {\begin{aligned}{\text{Attention}}(Q,K,V)={\text{softmax}}\left({\frac {QK^{\mathrm {T} }}{\sqrt {d_{k}}}}+sB\right)V\end{aligned}}}

Here,

s

{\displaystyle s}

is a real number ("scalar"), and

B

{\displaystyle B}

is the linear bias matrix defined by

B

=

(

0

1

2

3

■

_

1

0

1

2

■

–

2

–

1

0

1

■

–

3

–

2

–

1

0

■

■

■

■

■

■

)

{\displaystyle B={\begin{pmatrix}0&1&2&3&\cdots \\-1&0&1&2&\cdots \\-2&-1&0&1&\cdots \\-3&-2&-1&0&\cdots \\\vdots &\vdots &\vdots &\vdots &\ddots \\\end{pmatrix}}}

in other words,

B

i

,

j

=

j

−

i

{\displaystyle B_{i,j}=j-i}

. The idea being that the linear bias matrix is a softened mask. Just as

0

{\displaystyle 0}

represent full attention paid, and

−

∞

{\displaystyle -\infty }

represents no attention paid, the linear bias matrix increases attention paid in one direction and decreases attention paid in the other direction.

ALiBi allows pretraining on short context windows, then fine-tuning on longer context windows. Since it is directly plugged into the attention mechanism, it can be combined with any positional encoder that is plugged into the "bottom" of the entire network (which is where the sinusoidal encoder on the original transformer, as well as RoPE and many others, are located).

Relative Position Encodings

Relative Position Encodings is similar to ALiBi, but more generic:

Attention

(

Q

,

K

,

V

)

=

softmax

(

Q

K

T

d

k

+

B

)

V

{\displaystyle {\begin{aligned}{\text{Attention}}(Q,K,V)={\text{softmax}}\left({\frac {QK^{\mathrm {T} }}{\sqrt {d_{k}}}}+B\right)V\end{aligned}}}

where

B

{\displaystyle B}

is a Toeplitz matrix, that is,

B

i

,

j

=

B

i

′

,

j

′

{\displaystyle B_{i,j}=B_{i',j'}}

whenever

i

−

j

=

i

′

−

j

′

{\displaystyle i-j=i'-j'}

. This is contrasted with the original sinusoidal positional encoding, which is an "absolute positional encoding".

Efficient implementation

The transformer model has been implemented in standard deep learning frameworks such as TensorFlow and PyTorch. Transformers is a library produced by Hugging Face that supplies transformer-based architectures and pretrained models.

KV caching

When an autoregressive transformer is used for inference, such as generating text, the query vector is different at each step, but the already-computed key and value vectors are always the same. The KV caching method saves the computed key and value vectors at each attention block, so that they are not recomputed at each new token. PagedAttention applies memory paging to KV caching.

If a transformer is used with a baked-in prompt, such as ["You are a customer support agent..."], then the key and value vectors can be computed for the prompt, and saved on disk. The saving in compute is significant when the model is used for many short real-time interactions, such as in online chatbots.

In general, when a user uses an autoregressive transformer to generate a continuation to a sequence of tokens, the model would first perform a forward-pass on this sequence, whereby the KV caches over this sequence are computed. This is called prefilling. Hyperscalers serving large Transformer models may use disaggregated inference, wherein prefilling and decoding are performed on separately specialized hardware.

Continuous Batching

When using autoregressive transformer models, requests can have different input lengths and can finish generating at different times. Continuous batching adds new requests to batch and removes completed requests between decoding iterations instead of waiting for every request in a batch to finish. This improves hardware utilization and can reduce latency and increase throughput for systems that serve many concurrent requests.

Quantization

Transformer models are often quantized for inference, meaning that weights, activations, or both are represented with lower-precision numerical formats than the floating point formats used during training. Quantization can reduce memory use and data transfer costs and can improve inference speed on hardware that supports low precision arithmetic. The trade off is that aggressive quantization can come at the cost of model accuracy or may need calibration and specialized quantization methods to perform well.

FlashAttention

FlashAttention is an algorithm that implements the transformer attention mechanism efficiently on a GPU. It is a communication-avoiding algorithm that performs matrix multiplications in blocks, such that each block fits within the cache of a GPU, and by careful management of the blocks it minimizes data copying between GPU caches (as data movement is slow).

The FlashAttention method is a communication-avoiding algorithm that fuses these operations into a single loop, increasing the arithmetic intensity. It is an online algorithm that computes the following quantities:

z

i

=

q

T

k

i

m

i

=

max

(

z

1

,

…

,

z

i

)

=

max

(

m

i

–

1

,

z

i

)

■

i

=

e

z

1

–

m

i

+

■

+

e

z

i

–

m

i

=

e

m

i

–

1

–

m

i

■

i

–

1

+

e

z

i

–

m

i

o

i

=

e

z

1

–

m

i

v

1

+

■

+

e

z

i

–

m

i

v

i

=

e

m

i

–

1

–

m

i

o

i

–

1

+

e

z

i

–

m

i

v

i

{\displaystyle {\begin{aligned}z_{i}&=q^{T}k_{i}&\\m_{i}&=\max(z_{1},\dots
,z_{i})&={}&\max(m_{i-1},z_{i})\\\ell _{i}&=e^{z_{1}-m_{i}}+\dots
+e^{z_{i}-m_{i}}&={}&e^{m_{i-1}-m_{i}}\ell _{i-1}+e^{z_{i}-m_{i}}\\o_{i}&=e^{z_{1}-m_{i}}v_{1}+\dots
+e^{z_{i}-m_{i}}v_{i}&={}&e^{m_{i-1}-m_{i}}o_{i-1}+e^{z_{i}-m_{i}}v_{i}\end{aligned}}}

and returns

o

N

/

■

N

{\displaystyle o_{N}/\ell _{N}}

. In practice, FlashAttention operates over multiple queries and keys per loop iteration, in a similar way as blocked matrix multiplication. If backpropagation is needed, then the output vectors and the intermediate arrays

[

m

1

,

…

,

m

N

]

,

[

■

1

,

…

,

■

N

]

{\displaystyle [m_{1},\dots ,m_{N}],[\ell _{1},\dots ,\ell _{N}]}

are cached, and during the backward pass, attention matrices are rematerialized from these, making it a form of gradient checkpointing.

An improved version, FlashAttention-2, was developed to handle longer context lengths.

Multi-Query Attention

Multi-Query Attention changes the Multihead Attention mechanism. Whereas normally,

MultiheadAttention

(

Q

,

K

,

V

)

=

Concat

i

∈

[

n

heads

]

(

Attention

(

X

W

i

Q

,

X

W

i

K

,

X

W

i

V

)

)

W

O

{\displaystyle {\text{MultiheadAttention}}(Q,K,V)={\text{Concat}}_{i\in [n_{\text{heads}}]}\left({\text{Attention}}(XW_{i}^{Q},XW_{i}^{K},XW_{i}^{V})\right)W^{O}}

with Multi-Query Attention, there is just one

W

K

,

W

V

{\displaystyle W^{K},W^{V}}

, thus:

MultiQueryAttention

(

Q

,

K

,

V

)

=

Concat

i

∈

[

n

heads

]

(

Attention

(

X

W

i

Q

,

X

W

K

,

X

W

V

)

)

W

O

{\displaystyle {\text{MultiQueryAttention}}(Q,K,V)={\text{Concat}}_{i\in [n_{\text{heads}}]}\left({\text{Attention}}(XW_{i}^{Q},XW^{K},XW^{V})\right)W^{O}}

This has a neutral effect on model quality and training speed, but increases inference speed.

More generally, grouped-query attention (GQA) partitions attention heads into groups, each of which shares the key-value pair. MQA is GQA with one group, while standard Multihead Attention is GQA with the maximal number of groups.

Multihead Latent Attention (MLA) is a low-rank approximation to standard MHA. Specifically, each hidden vector, before entering the attention mechanism, is first projected to two low-dimensional spaces ("latent space"), one for query and one for key-value (KV vector). This design minimizes the KV cache, as only the low-dimensional KV vector needs to be cached.

Speculative decoding

Speculative decoding is a method to accelerate token decoding. Similarly to speculative execution in CPUs, future tokens are computed quickly, then verified. If the quickly computed tokens are incorrect, they are discarded and computed slowly.

The key factor in speculative decoding is that a transformer decoder can verify faster than it can decode, in the following sense.

Suppose we have two transformer models like GPT-3 and GPT-3-small, both with a context window size of 512. To generate an entire context window autoregressively with greedy decoding with GPT-3, it must be run for 512 times, each time generating a token

x

1

,

x

2

,

.

.

.

,

x

512

{\displaystyle x_{1},x_{2},...,x_{512}}

, taking time

512

T

Gpt-3

{\displaystyle 512T_{\text{GPT-3}}}

. However, if we had some educated guess for the values of these tokens, we could verify all of them in parallel, in one run of the model, by checking that each

x

t

{\displaystyle x_{t}}

is indeed the token with the largest log-likelihood in the

t

{\displaystyle t}

-th output.

In speculative decoding, a smaller model or some other simple heuristic is used to generate a few speculative tokens that are subsequently verified by the larger model. For example, suppose we use GPT-3-small to generate four speculative tokens:

x

~

1

,

x

~

2

,

x

~

3

,

x

~

4

{\displaystyle {\tilde {x}}_{1},{\tilde {x}}_{2},{\tilde {x}}_{3},{\tilde {x}}_{4}}

. This only takes

4

T

GPT-3-small

{\displaystyle 4T_{\text{GPT-3-small}}}

. These tokens are then run through the larger GPT-3 in one go. Suppose that

x

~

1

{\displaystyle {\tilde {x}}_{1}}

and

x

~

2

{\displaystyle {\tilde {x}}_{2}}

are verified by GPT-3 as what it would have picked, then those are kept, but

x

~

3

{\displaystyle {\tilde {x}}_{3}}

is not, so

x

~

3

,

x

~

4

{\displaystyle {\tilde {x}}_{3},{\tilde {x}}_{4}}

are discarded, and GPT-3 is run on those. This would take

4

T

GPT-3-small

+

3

T

Gpt-3

{\displaystyle 4T_{\text{GPT-3-small}}+3T_{\text{GPT-3}}}

, which might be shorter than

4

T

Gpt-3

{\displaystyle 4T_{\text{GPT-3}}}

.

For non-greedy decoding, similar ideas apply, except the speculative tokens are accepted or rejected stochastically, in a way that guarantees the final output distribution is the same as if speculative decoding was not used.

In Multi-Token Prediction, a single forward pass creates a final embedding vector, which then is un-embedded into a token probability. However, that vector can then be further processed by another transformer block to predict the next token, and so on for arbitrarily many steps into the future. This trades off accuracy for speed, since each new token costs just one more transformer block, rather than the entire stack.

Sub-quadratic transformers

Training transformer-based architectures can be expensive, especially for long inputs. Many methods have been developed to attempt to address the issue. In the image domain, Swin transformer is an efficient architecture that performs attention inside shifting windows. In the audio domain, SepTr decouples the attention in time and frequency domains. Long Range Arena (2020) is a standard benchmark for comparing the behavior of transformer architectures over long inputs.

Alternative attention graphs

The standard attention graph is either all-to-all or causal, both of which scales as

O

(

N

2

)

{\displaystyle O(N^{2})}

where

N

{\displaystyle N}

is the number of tokens in a sequence.

Reformer (2020) reduces the computational load from

O

(

N

2

)

{\displaystyle O(N^{2})}

to

O

(

N

ln

■

N

)

{\displaystyle O(N\ln N)}

by using locality-sensitive hashing and reversible layers.

Sparse attention uses attention graphs that grows slower than

O

(

N

2

)

{\displaystyle O(N^{2})}

. For example, BigBird (2020) uses random small-world networks which grows as

O

(

N

)

{\displaystyle O(N)}

.

Ordinary transformers require a memory size that is quadratic in the size of the context window. Attention-free transformers reduce this to a linear dependence while still retaining the advantages of a transformer by linking the key to the value.

Random Feature Attention

Random Feature Attention (2021) uses Fourier random features:

ϕ

(

x

)

=

1

D

[
cos
■
■
w
1
,
x
■
,
sin
■
■
w
1
,
x
■
,
■
cos
■
■
w

D

,
x
■
,

sin

■

■

w

D

,

x

■

]

T

{\displaystyle \varphi (x)={\frac {1}{\sqrt {D}}}[\cos \langle w_{1},x\rangle ,\sin \langle w_{1},x\rangle ,\cdots \cos \langle w_{D},x\rangle ,\sin \langle w_{D},x\rangle]^{T}}

where

w

1

,

.

.

.

,

w

D

{\displaystyle w_{1},...,w_{D}}

are independent samples from the normal distribution

N

(

0

,

σ

2

I

)

{\displaystyle N(0,\sigma ^{2}I)}

. This choice of parameters satisfies

E

[

■

ϕ

(

x

)

,

ϕ

(

y

)

■

]

=

e

_

■

x

_

y

■

2

2

σ

2

{\displaystyle \mathbb {E} [\langle \varphi (x),\varphi (y)\rangle]=e^{-{\frac {\|x-y\|^{2}}{2\sigma ^{2}}}}}

, or

e

■

x

,

y

■

/

σ

2

=

E

[

■

e

■

x

■

2

/

2

σ

2

ϕ

(

x

)

,

e

■

y

■

2

/

2

σ

2

ϕ

(

y

)

■

]

≈

■

e

■

x

■

2

/

2

σ

2

ϕ

(

x

)

,

e

■

y

■

2

/

2

σ

2

ϕ

(

y

)

■

{\displaystyle e^{\langle x,y\rangle /\sigma ^{2}}=\mathbb {E} [\langle e^{\|x\|^{2}/2\sigma ^{2}}\varphi (x),e^{\|y\|^{2}/2\sigma ^{2}}\varphi (y)\rangle]\approx \langle e^{\|x\|^{2}/2\sigma ^{2}}\varphi (x),e^{\|y\|^{2}/2\sigma ^{2}}\varphi (y)\rangle }

Consequently, the one-headed attention, with one query, can be written as

Attention

(

q

,

K

,

V

)

=

softmax

(

q

K

T

d

k

)

V

≈

ϕ

(

q

)

T

Σ

i

e

■

k

i

■

2

/

2

σ

2

ϕ

(

k

i

)

v

i

T

ϕ

(

q

)

T

Σ

i

e

■

k

i

■

2

/

2

σ

2

ϕ

(

k

i

)

{\displaystyle {\text{Attention}}(q,K,V)={\text{softmax}}\left({\frac {qK^{\mathrm {T} }}{\sqrt {d_{k}}}}\right)V\approx {\frac {\varphi (q)^{T}\sum _{i}e^{\|k_{i}\|^{2}/2\sigma ^{2}}\varphi (k_{i})v_{i}^{T}}{\varphi (q)^{T}\sum _{i}e^{\|k_{i}\|^{2}/2\sigma ^{2}}\varphi (k_{i})}}}

where

σ

=

d

K

1

/

4

{\displaystyle \sigma =d_{K}^{1/4}}

. Similarly for multiple queries, and for multihead attention.

This approximation can be computed in linear time, as we can compute the matrix

ϕ

(

k

i

)

v

i

T

{\displaystyle \varphi (k_{i})v_{i}^{T}}

first, then multiply it with the query. In essence, we have managed to obtain a more precise version of

Attention

(

Q

,

K

,

V

)

=

softmax

(

Q

K

T

d

k

)

V

≈

Q

(

K

T

V

/

d

k

)

{\displaystyle {\text{Attention}}(Q,K,V)={\text{softmax}}\left({\frac {QK^{\mathrm {T} }}{\sqrt {d_{k}}}}\right)V\approx Q(K^{T}V/{\sqrt {d_{k}}})}

Performer (2022) uses the same Random Feature Attention, but

w

1

,

.

.

.

,

w

D

{\displaystyle w_{1},...,w_{D}}

are first independently sampled from the normal distribution

N

(

0

,

σ

2

I

)

{\displaystyle N(0,\sigma ^{2}I)}

, then they are Gram–Schmidt processed.

Multimodality

Transformers can also be used/adapted for modalities (input or output) beyond just text, usually by finding a way to "tokenize" the modality.

Multimodal models can either be trained from scratch, or by finetuning. A 2022 study found that transformers pretrained only on natural language can be finetuned on only 0.03% of parameters and become competitive with LSTMs on a variety of logical and visual tasks, demonstrating transfer learning. The LLaVA was a vision-language model composed of a language model (Vicuna-13B) and a vision model (ViT-L/14), connected by a linear layer. Only the linear layer is finetuned.

Vision transformers adapt the transformer to computer vision by breaking down input images as a series of patches, turning them into vectors, and treating them like embedding vectors of tokens in a standard transformer.

Conformer and later Whisper follow the same pattern for speech recognition, first turning the speech signal into a spectrogram, which is then treated like an image, i.e. broken down into a series of patches, turned into vectors and treated like embedding vector of tokens in a standard transformer.

Perceivers are a variant of transformers designed for multimodality.

For image generation, notable architectures are DALL-E 1 (2021), Parti (2022), Phenaki (2023), and Muse (2023). Unlike later models, DALL-E is not a diffusion model. Instead, it uses a decoder-only transformer that autoregressively generates a text, followed by the token representation of an image, which is then converted by a variational autoencoder to an image. Parti is an encoder–decoder transformer, where the encoder processes a text prompt, and the decoder generates a token representation of an image. Muse is an encoder-only transformer that is trained to predict masked image tokens from unmasked image tokens. During generation, all input tokens are masked, and the highest-confidence predictions are included for the next iteration, until all tokens are predicted. Phenaki is a text-to-video model. It is a bidirectional masked transformer conditioned on pre-computed text tokens. The generated tokens are then decoded to a video.

Applications

The transformer has had great success in natural language processing (NLP). Many large language models such as GPT-2, GPT-3, GPT-4, Gemini, AlbertAGPT, Claude, BERT, Grok, XLNet, RoBERTa and ChatGPT demonstrate the ability of transformers to perform a wide variety of NLP-related subtasks and their related real-world applications, including:

machine translation

time series prediction

document summarization

document generation

named entity recognition (NER)

writing computer code based on requirements expressed in natural language.

speech-to-text

Beyond traditional NLP, the transformer architecture has had success in other applications, such as:

biological sequence analysis

video understanding

protein folding (such as AlphaFold)

evaluating chess board positions. Using static evaluation alone (that is, with no Minimax search) transformer achieved an Elo of 2895, putting it at grandmaster level.

service function chain embedding

See also

seq2seq – Family of machine learning approaches

Circuit (neural network) – Interpretable computational sub-graphs within artificial neural networks

Perceiver – Variant of Transformer designed for multimodal data

Vision transformer – Machine learning model for vision processing

Large language model – Type of machine learning model

BERT (language model) – Series of language models developed by Google AI

Generative pre-trained transformer – Type of large language model

T5 (language model) – Series of large language models developed by Google AI

Notes

References

Further reading